PYTHON IMPORTANT QUESTIONS

UNIT - 1

1 (a) Discuss the int(), float(), str() and complex() type conversion functions with examples

In Python, **type conversion** means converting one data type into another using built-in functions. The common type conversion functions are **int(), float(), str(), and complex()**.

1. int() Function

The **int()** function converts a value into an **integer**.

Syntax

int(x)

Examples

a = int(5.8)
print(a)

Output: 5

b = int("20")
print(b)

Output: 20

It removes the decimal part of a floating number and converts numeric strings into integers.

---

2. float() Function

The **float()** function converts a value into a **floating point number**.

Syntax

float(x)

Examples

a = float(10)
print(a)

Output: 10.0

b = float("3.5")
print(b)

Output: 3.5

It converts integers or numeric strings into decimal numbers.

---

3. str() Function

The **str()** function converts a value into a **string**.

Syntax

str(x)

Examples

a = str(100)
print(a)

Output: "100"

It is mainly used to convert numbers into strings for printing or concatenation.

Example:

age = 20
print("Age is " + str(age))

---

4. complex() Function

The **complex()** function converts numbers into **complex numbers** having a real and imaginary part.

Syntax

complex(real, imaginary)

Example

```
a = complex(2,3)
print(a)
```

Output: (2+3j)

Here **2 is the real part** and **3j is the imaginary part**.

Conclusion:
These type conversion functions help in converting data from one type to another, making Python programs flexible and easy to use.

---

1 (b) Write a program to print the sum of the series

1 + 1/2 + 1/3 + ... + 1/n

The given series is called the **harmonic series**. The program calculates the sum of terms from **1 to n**.

Python Program

```
n = int(input("Enter value of n: "))
sum = 0

for i in range(1, n+1):
  sum = sum + 1/i

print("Sum of series =", sum)
```

Explanation

- The value of **n** is taken from the user.
- A loop runs from **1 to n**.
- Each term **1/i** is added to the variable sum.
- Finally, the total sum of the series is printed.

Example:
If n = 4

Series =
1 + 1/2 + 1/3 + 1/4

Sum ≈ **2.0833**.

2 (a) Describe Membership and Identity Operators with suitable examples

In Python, **operators** are symbols used to perform operations on variables and values. Two important types are **Membership operators** and **Identity operators**.

1. Membership Operators

Membership operators are used to **check whether a value exists in a sequence** such as a list, tuple, or string.

The two membership operators are:

- **in**
- **not in**

Example

```
a = [10, 20, 30, 40]

print(20 in a)
print(50 not in a)
```

Output

True
True

Explanation

- 20 in a checks whether **20 is present in the list**.
- 50 not in a checks whether **50 is not present in the list**.

Another example:

```
name = "Python"
print("P" in name)
```

Output: True

---

2. Identity Operators

Identity operators are used to **check whether two variables refer to the same memory location**.

The identity operators are:

- **is**
- **is not**

Example

```
a = 10
b = 10

print(a is b)
```

Output

True

This means both variables refer to the **same object in memory**.

Another example:

```
x = [1,2,3]
y = [1,2,3]

print(x is y)
```

Output:

False

Although the values are the same, they are **stored in different memory locations**.

Conclusion:
Membership operators check **whether an element exists in a sequence**, while identity operators check **whether two variables refer to the same object in memory**.

---

2 (b) Python program to read four integers and print them without space or newline

This program reads four integers from the user and prints them using a **single print statement** without spaces or newline.

Python Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
d = int(input("Enter fourth number: "))

print(a, b, c, d, sep="", end="")
```

Explanation

- The program reads **four integers** using input().
- sep="" removes the **space between numbers**.
- end="" removes the **newline at the end**.

Example Output

If the input values are:

```
1
2
3
4
```

Output:

1234

3 (a) Explain with an example while loop, break statement and continue statement in Python

1. While Loop

A **while loop** is used to execute a block of code repeatedly as long as a given condition is **true**.

Syntax

```
while condition:
  statements
```

Example

```
i = 1
while i <= 5:
  print(i)
  i += 1
```

Output

1
2
3
4
5

The loop runs until the condition i <= 5 becomes false.

---

2. Break Statement

The **break statement** is used to **terminate the loop immediately**, even if the condition is still true.

Example

```
for i in range(1, 10):
  if i == 5:
    break
  print(i)
```

Output

1
2
3
4

When i becomes **5**, the loop stops.

---

3. Continue Statement

The **continue statement** is used to **skip the current iteration** and continue with the next iteration of the loop.

Example

```
for i in range(1, 6):
  if i == 3:
    continue
  print(i)
```

Output

1
2
4
5

When i = 3, that iteration is skipped.

Conclusion:

- **while loop** repeats statements while a condition is true.
- **break** stops the loop completely.
- **continue** skips the current iteration and moves to the next one.

---

3 (b) Python program to find numbers divisible by 2 and multiples of 3 between 1 and 500

A number divisible by **2 and 3** means it is divisible by **6**.

Python Program

```
for i in range(1, 501):
  if i % 2 == 0 and i % 3 == 0:
    print(i)
```

Explanation

- The loop runs from **1 to 500**.
- i % 2 == 0 checks if the number is **divisible by 2**.
- i % 3 == 0 checks if the number is **divisible by 3**.
- If both conditions are true, the number is printed.

Example Output

```
6
12
18
24
30
...
498
```

These numbers are **divisible by 2 and multiples of 3**.

4 (a) Define the following with necessary examples

i. if Statement

The **if statement** is used to execute a block of code **only when a given condition is true**.

Syntax

```
if condition:
  statements
```

Example

```
num = 10

if num > 5:
  print("Number is greater than 5")
```

Output

Number is greater than 5

Explanation:
The condition num > 5 is true, so the statement inside the if block is executed.

---

ii. If-elif-else Statement

The **if-elif-else statement** is used when we want to **check multiple conditions**.
Python checks the conditions one by one and executes the block whose condition is true.

Syntax

```
if condition1:
  statements
elif condition2:
  statements
else:
  statements
```

Example

```
marks = 75

if marks >= 90:
  print("Grade A")
```

```
elif marks >= 60:
   print("Grade B")
else:
   print("Grade C")
```

Output

Grade B

Explanation:
Since the marks are **75**, the condition marks >= 60 becomes true, so **Grade B** is printed.

---

4 (b) Python program to convert height in feet and inches to cm

Formula

- **1 foot = 12 inches**
- **1 inch = 2.54 cm**

Python Program

```
feet = int(input("Enter height in feet: "))
inch = int(input("Enter height in inches: "))

total_inches = feet * 12 + inch
cm = total_inches * 2.54

print("Height in cm =", cm)
```

Explanation

1. The program reads height in **feet and inches**.
2. Feet are converted into **inches** using feet * 12.
3. Total inches are converted into **centimeters** using × 2.54.
4. The result is printed.

Sample Input

2 feet 7 inches

Output

78.74 cm

(a) List out and explain the key features of Python programming

Python is a **high-level, interpreted programming language** known for its simplicity and readability. Some important features of Python are:

1. Easy to Learn and Use

Python has a **simple and readable syntax**, making it easy for beginners to learn and understand.

Example

```
print("Hello World")
```

---

2. Interpreted Language

Python is an **interpreted language**, meaning the code is executed line by line without compiling the entire program.

---

3. Object-Oriented

Python supports **object-oriented programming (OOP)** concepts such as classes, objects, inheritance, and polymorphism.

---

4. Portable

Python programs can run on **different platforms** such as Windows, Linux, and Mac without modification.

---

5. Large Standard Library

Python provides a **large collection of built-in libraries** that help in tasks like file handling, web development, and data analysis.

---

6. Dynamically Typed

In Python, the **data type of a variable is decided automatically** during execution.

Example:

```
a = 10
a = "Python"
```

---

7. Open Source

Python is **free and open source**, meaning anyone can download, use, and modify it.

Conclusion:
Because of its simplicity, portability, and powerful libraries, Python is widely used in **web development, data science, artificial intelligence, and software development**.

---

5 (b) Python program to print numbers from 0 to 100 except multiples of 4 and 6

This program prints numbers from **0 to 100** but skips numbers that are **multiples of 4 or 6**.

Python Program

```
for i in range(0, 101):
  if i % 4 == 0 or i % 6 == 0:
    continue
  print(i)
```

Explanation

- The loop runs from **0 to 100**.
- i % 4 == 0 checks if the number is a **multiple of 4**.
- i % 6 == 0 checks if the number is a **multiple of 6**.
- If either condition is true, the **continue statement skips that number**.
- Otherwise, the number is printed.

Example Output (first few numbers):

```
1
2
3
5
7
9
10
11
13
...
```

These numbers are printed **except multiples of 4 and 6**.

6 (a) Explain the execution behavior of 'for' loop in Python with an example

The **for loop** in Python is used to **repeat a block of statements for each item in a sequence** such as a list, tuple, string, or range of numbers.

Syntax

```
for variable in sequence:
  statements
```

Execution Behavior

1. The loop takes the **first value from the sequence**.
2. The statements inside the loop are **executed**.
3. The variable takes the **next value in the sequence**.
4. The process continues **until all values are processed**.
5. When the sequence ends, the loop **terminates automatically**.

Example Program

```
for i in range(1, 6):
  print(i)
```

Output

1
2
3
4
5

Explanation

- range(1,6) generates numbers from **1 to 5**.
- The loop variable i takes each value and prints it.
- After printing **5**, the loop stops.

Conclusion:
The for loop is mainly used when the **number of iterations is known in advance**.

---

6 (b) Python program to demonstrate loop control statements

Loop control statements are used to **change the normal execution of a loop**. The main loop control statements in Python are:

- **break**
- **continue**
- **pass**

Python Program

```
for i in range(1, 10):

  if i == 3:
    continue   # skips number 3

  if i == 7:
    break     # stops the loop at 7

  if i == 5:
    pass      # does nothing

  print(i)
```

Output

1
2
4
5
6

Explanation

- **continue** skips the current iteration (3 is skipped).
- **pass** does nothing but allows the program to continue.
- **break** terminates the loop when i becomes 7.

Conclusion:
Loop control statements help in **controlling the flow of loops**, making programs more flexible and efficient.

7 (a) Explain how input and output functions are performed in Python with an example

In Python, **input and output functions** are used to interact with the user.

- **Input functions** are used to take data from the user.
- **Output functions** are used to display the result.

1. Input Function

The **input()** function is used to take input from the user through the keyboard.
The input value is stored in a variable.

Syntax

variable = input("message")

Example

```
name = input("Enter your name: ")
print("Hello", name)
```

Output

```
Enter your name: Kavitha
Hello Kavitha
```

---

2. Output Function

The **print()** function is used to display output on the screen.

Example

```
a = 10
b = 20
print("Sum =", a + b)
```

Output

```
Sum = 30
```

Conclusion:
The input() function is used to **receive data from the user**, while the print() function is used to **display the result on the screen**.

---

7 (b) Python program to check whether a number is an Armstrong number or not

An **Armstrong number** is a number that is equal to the **sum of the cubes of its digits**.

Example:
$153 = 1^3 + 5^3 + 3^3 = 153$

Python Program

```
num = int(input("Enter a number: "))

temp = num
sum = 0

while num > 0:
  digit = num % 10
  sum = sum + digit**3
  num = num // 10

if temp == sum:
  print("Armstrong Number")
else:
  print("Not an Armstrong Number")
```

Explanation

- The program reads a number from the user.
- Each digit is separated using % 10.
- The cube of each digit is added to sum.
- If the result equals the original number, it is an **Armstrong number**.

Example

Input:

153

Output:

Armstrong Number

8 (a) List and Explain Python Loop Control Statements with examples

Loop control statements are used to **control the flow of loops** in Python. They change the normal execution of a loop.

The main loop control statements are:

1. break Statement

The **break** statement is used to **terminate the loop immediately** when a condition is satisfied.

Example

```
for i in range(1, 10):
  if i == 5:
    break
  print(i)
```

Output

1
2
3
4

When i = 5, the loop stops.

---

2. continue Statement

The **continue** statement is used to **skip the current iteration** and continue with the next iteration of the loop.

Example

```
for i in range(1, 6):
  if i == 3:
    continue
  print(i)
```

Output

1
2
4
5

The value **3 is skipped**.

---

3. pass Statement

The **pass** statement is a **null statement**. It does nothing but is used when a statement is required syntactically.

Example

```
for i in range(1, 4):
  pass
print("Loop executed")
```

Output

Loop executed

Conclusion:
Loop control statements such as **break, continue, and pass** help in controlling the execution of loops and make programs more flexible.

---

8 (b) Python program to display Fibonacci sequence up to nth term

The **Fibonacci sequence** is a series of numbers where each number is the **sum of the previous two numbers**.

Example:
0, 1, 1, 2, 3, 5, 8, 13 ...

Python Program

```
n = int(input("Enter number of terms: "))

a = 0
b = 1

print("Fibonacci Series:")

for i in range(n):
  print(a)
  c = a + b
  a = b
  b = c
```

Explanation

- a and b represent the first two numbers of the series.
- Each next term is calculated using c = a + b.
- The loop runs **n times** to display the Fibonacci numbers.

Example Output

Input:

5

Output:

0
1
1
2
3

9 (a) Explain the Program Development Life Cycle with neat sketch

The **Program Development Life Cycle (PDLC)** is a step-by-step process used to develop a computer program. It helps programmers design, write, test, and maintain programs efficiently.

Steps in Program Development Life Cycle

1. Problem Definition

In this step, the **problem is clearly identified and understood**.
The programmer determines **what the program should do**.

2. Program Design

The solution is planned using **algorithms, flowcharts, or pseudocode**.

3. Coding

In this step, the algorithm is **converted into a program using a programming language** such as Python.

4. Testing and Debugging

The program is **tested for errors**.
If errors are found, they are **corrected (debugging)**.

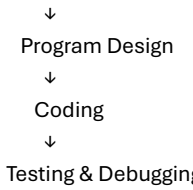
5. Documentation

Documentation explains **how the program works**, including comments and user guides.

6. Maintenance

After implementation, the program may need **updates, improvements, or bug fixes**.

Neat Sketch (PDLC)



↓

Documentation

↓

Maintenance

Conclusion:
The Program Development Life Cycle ensures that programs are **developed systematically and efficiently**.

---

9 (b) Python program to check loan eligibility

Python Program

```
name = input("Enter Applicant Name: ")
age = int(input("Enter Age: "))
income = int(input("Enter Monthly Income: "))
credit = int(input("Enter Credit Score: "))

if age >= 21 and age <= 60 and income >= 25000 and credit >= 700:
  print("Loan Approved")

elif age >= 21 and age <= 60 and income >= 25000 and credit >= 650 and
credit < 700:
  print("Loan Approved with Conditions")

else:
  print("Loan Rejected")
```

Explanation

1. The program reads **name, age, monthly income, and credit score** from the user.
2. It checks the **loan eligibility conditions**:
    - Age between **21 and 60**
    - Income **≥ 25,000**
    - Credit score **≥ 700**
3. If all conditions are satisfied, **Loan Approved** is printed.
4. If credit score is **650–699**, the loan is **approved with conditions**.
5. If any major condition fails, the loan is **rejected**.

Example Output

```
Enter Applicant Name: Ravi
Enter Age: 30
Enter Monthly Income: 40000
Enter Credit Score: 720

Loan Approved
```

---

10 (a) Write and explain the applications of Python

Python is a **high-level, general-purpose programming language** used in many fields because of its simplicity and powerful libraries.

Applications of Python

1. Web Development

Python is widely used to develop websites and web applications using frameworks such as **Django and Flask**.

2. Data Science and Data Analysis

Python is used for **data analysis, data visualization, and statistics** using libraries like **NumPy, Pandas, and Matplotlib**.

3. Artificial Intelligence and Machine Learning

Python is popular in **AI and Machine Learning** because of libraries such as **TensorFlow, Keras, and Scikit-learn**.

4. Automation and Scripting

Python is used to **automate repetitive tasks** like file handling, system operations, and testing.

5. Game Development

Python can be used to create games using libraries such as **Pygame**.

6. Desktop Applications

Python is used to develop desktop software using tools like **Tkinter and PyQt**.

7. Scientific Computing

Python is widely used in **research and scientific calculations** because it supports mathematical and scientific libraries.

Conclusion:
Due to its simplicity, portability, and rich libraries, Python is widely used in **software development, data science, artificial intelligence, and automation**.

---

10 (b) Python program to calculate electricity bill

The electricity bill is calculated based on **different slab rates** and a **fixed meter charge of ₹100**.

Python Program

```
units = int(input("Enter number of units consumed: "))

bill = 0

if units <= 100:
    bill = units * 1.5

elif units <= 200:
    bill = (100 * 1.5) + (units - 100) * 2.5

elif units <= 300:
    bill = (100 * 1.5) + (100 * 2.5) + (units - 200) * 4

else:
    bill = (100 * 1.5) + (100 * 2.5) + (100 * 4) + (units - 300) * 6

bill = bill + 100   # fixed meter charge

print("Electricity Bill = ₹", bill)
```

Explanation

1. The program reads the **number of units consumed** from the user.
2. Different **rates are applied for each slab range**.
3. The bill is calculated step by step according to the slabs.
4. A **fixed meter charge of ₹100** is added to the total bill.
5. Finally, the electricity bill is displayed.

Example Output

Enter number of units consumed: 250
Electricity Bill = ₹ 800

PYTHON IMPORTANT QUESTIONS

UNIT – 2

1 (a) Python List Built-in Methods with Program

Python List

A **list** in Python is an ordered and mutable collection used to store multiple items in a single variable.

Common List Built-in Methods

Method	Description
append()	Adds an element to the end of the list
insert()	Inserts an element at a specified position
remove()	Removes a specific element
pop()	Removes an element using index
sort()	Sorts the list
reverse()	Reverses the list order
count()	Returns number of occurrences
index()	Returns position of an element
clear()	Removes all elements from the list

Python Program

```
# Demonstration of list methods

numbers = [10, 20, 30, 40]

numbers.append(50)
print("After append:", numbers)

numbers.insert(2, 25)
print("After insert:", numbers)

numbers.remove(30)
print("After remove:", numbers)

numbers.pop()
print("After pop:", numbers)

numbers.sort()
print("Sorted list:", numbers)

numbers.reverse()
print("Reversed list:", numbers)

print("Index of 20:", numbers.index(20))
print("Count of 40:", numbers.count(40))
```

---

1 (b) Python Program to Print Even Numbers from a List

An **even number** is a number divisible by **2**.

Program

```
# List of numbers
list1 = [12, 17, 20, 25, 30, 33]

print("Even numbers are:")

for i in list1:
  if i % 2 == 0:
    print(i)
```

Output

```
Even numbers are:
12
```

20
30

Explanation:
The program checks each number in the list. If the number is divisible by **2**, it prints the number as an even number.

---

2 (a) Explain the steps to create a Dictionary in Python and list out Dictionary Built-in functions

Dictionary in Python

A **dictionary** is a collection of **key–value pairs** used to store data values. Dictionaries are **mutable, unordered, and indexed by keys**.

Steps to Create a Dictionary

1. Use **curly braces { }** to create a dictionary.
2. Insert elements as **key : value pairs**.
3. Keys must be **unique**, while values can be duplicated.
4. Access elements using **keys**.

Example

```
student = {
  "name": "Ravi",
  "age": 20,
  "course": "B.Tech"
}

print(student["name"])
```

Output:

Ravi

Dictionary Built-in Functions / Methods

Method	Description
keys()	Returns all keys in the dictionary
values()	Returns all values
items()	Returns key-value pairs
get()	Returns value of a specified key
update()	Updates dictionary elements
pop()	Removes a specified key
popitem()	Removes last inserted item
clear()	Removes all elements
copy()	Returns a copy of the dictionary

---

2 (b) Python Program to Perform Linear Search on a List

Linear Search

Linear search is a simple searching technique where each element of the list is checked one by one until the required element is found.

Algorithm

1. Start from the first element of the list.
2. Compare each element with the target value.
3. If the element matches, return its position.
4. If the end of the list is reached, the element is not found.

Python Program

```
# Linear Search Program

list1 = [10, 25, 30, 45, 50]
```

```
key = int(input("Enter number to search: "))

found = False

for i in range(len(list1)):
    if list1[i] == key:
        print("Element found at position", i+1)
        found = True
        break

if not found:
    print("Element not found")
```

Example Output

```
Enter number to search: 30
Element found at position 3
```

3 (a) What is Immutable Data Type? Explain Various String Operations with Program

Immutable Data Type

An **immutable data type** is a data type whose value **cannot be changed after it is created**. If we try to modify it, Python creates a **new object** instead of changing the original one.

Examples

- int
- float
- string
- tuple

A **string** is an immutable data type in Python.

String Operations in Python

Operation	Description	Example
Concatenation	Joining two strings	"Hello" + "World"
Repetition	Repeating a string	"Hi"*3
Indexing	Accessing characters	s[0]
Slicing	Extracting part of string	s[1:4]
Length	Finds number of characters	len(s)
Upper/Lower	Change case	s.upper()

Python Program

```
s = "Python"

# Concatenation
print("Concatenation:", s + " Programming")

# Repetition
print("Repetition:", s * 2)

# Indexing
print("First character:", s[0])

# Slicing
print("Slice:", s[1:4])

# Length
print("Length:", len(s))

# Uppercase
print("Upper:", s.upper())
```

Output

Concatenation: Python Programming
Repetition: PythonPython
First character: P
Slice: yth
Length: 6
Upper: PYTHON

---

3 (b) Python Program to Find Addition of Two Matrices Using List

Matrix addition means **adding corresponding elements of two matrices**.

Python Program

```
# Matrix A
A = [[1, 2, 3],
   [4, 5, 6]]

# Matrix B
B = [[7, 8, 9],
   [1, 2, 3]]

result = [[0,0,0],
       [0,0,0]]

# Matrix addition
for i in range(len(A)):
  for j in range(len(A[0])):
    result[i][j] = A[i][j] + B[i][j]

# Display result
print("Result Matrix:")
for r in result:
  print(r)
```

Output

Result Matrix:
[8, 10, 12]
[5, 7, 9]

Explanation

- Two matrices are stored as **lists of lists**.
- Nested **for loops** are used to add corresponding elements.
- The result is stored in a **result matrix**.

---

4 (a) Is Set an Ordered Collection? Justify with Examples and Explain Set Operations

Set in Python

A **set** in Python is a collection of **unique elements** enclosed in **curly braces { }**.

Example:

A = {1, 2, 3, 4}

Is Set an Ordered Collection?

No, **set is not an ordered collection**.
Elements in a set **do not have a fixed position**, and the order may change whenever it is printed.

Example

```
A = {10, 20, 30, 40}
print(A)
```

Output may be:

{40, 10, 20, 30}

This shows that **sets do not maintain insertion order**.

Also, sets **do not allow duplicate values**.

Example:

A = {1, 2, 2, 3}
print(A)

Output:

{1, 2, 3}

Set Operations

Operation	Description	Example
Union	Combines elements of two sets	`A
Intersection	Common elements in both sets	A & B
Difference	Elements in A but not in B	A - B
Symmetric Difference	Elements in either set but not both	A ^ B
Add	Adds an element to set	A.add(5)
Remove	Removes an element	A.remove(2)

Example Program

```
A = {1, 2, 3}
B = {3, 4, 5}

print("Union:", A | B)
print("Intersection:", A & B)
print("Difference:", A - B)
print("Symmetric Difference:", A ^ B)
```

---

4 (b) Python Program to Find Trace of a Matrix Using Tuple

Trace of a Matrix

The **trace of a matrix** is the **sum of the elements in the main diagonal**.

Example:

Matrix

1 2 3
4 5 6
7 8 9

Trace = **1 + 5 + 9 = 15**

Python Program

```
# Matrix stored as tuple
matrix = ((1, 2, 3),
          (4, 5, 6),
          (7, 8, 9))

trace = 0

for i in range(len(matrix)):
    trace = trace + matrix[i][i]

print("Trace of matrix =", trace)
```

Output

Trace of matrix = 15

Explanation

- Matrix is stored using **tuple of tuples**.
- The **main diagonal elements** are accessed using matrix[i][i].
- Their sum gives the **trace of the matrix**.

5 (a) Does mutability support for List? Explain any two methods with example.

Yes, **lists in Python support mutability**.
Mutability means the elements of a list **can be changed, added, or removed after the list is created** without creating a new object.

Example:

```
list1 = [10, 20, 30]
list1[1] = 25
print(list1)
```

Output:

[10, 25, 30]

This shows that the list element is **modified directly**.

Two List Methods

1. append()

The append() method **adds an element to the end of the list**.

Example:

```
list1 = [1, 2, 3]
list1.append(4)
print(list1)
```

Output:

[1, 2, 3, 4]

---

2. remove()

The remove() method **removes a specified element from the list**.

Example:

```
list1 = [10, 20, 30, 40]
list1.remove(20)
print(list1)
```

Output:

[10, 30, 40]

Thus, lists are **mutable because elements can be modified using different methods**.

---

5 (b) Distinguish between

i) Lists and Tuples

Lists	**Tuples**
Mutable (can be changed)	Immutable (cannot be changed)
Created using [ ]	Created using ( )
Elements can be added or removed	Elements cannot be modified
Example: [1,2,3]	Example: (1,2,3)
Used when data needs modification	Used when data should remain constant

---

ii) Sets and Dictionaries

Sets	**Dictionaries**
Collection of unique elements	Collection of key-value pairs
Created using { }	Created using {key:value}
Do not allow duplicate values	Keys must be unique
No indexing	Accessed using keys
Example: {1,2,3}	Example: {"name":"Ram", "age":20}

---

6 (a) Explain Negative Indexing and List Slicing in Python with Program

Negative Indexing

In Python, **negative indexing** is used to access elements from the **end of a list**.
The last element has index **-1**, the second last **-2**, and so on.

Example:

```
list1 = [10, 20, 30, 40, 50]
print(list1[-1])
print(list1[-2])
```

Output

```
50
40
```

Here -1 refers to the **last element** and -2 refers to the **second last element**.

List Slicing

List slicing is used to **extract a part of a list**.
The syntax is:

```
list[start : end : step]
```

- **start** – starting index
- **end** – ending index (not included)
- **step** – increment value

Example Program

```
list1 = [10, 20, 30, 40, 50, 60]

print("Original List:", list1)

print("First three elements:", list1[0:3])

print("Middle elements:", list1[2:5])

print("Using negative indexing:", list1[-4:-1])
```

Output

```
Original List: [10, 20, 30, 40, 50, 60]
First three elements: [10, 20, 30]
Middle elements: [30, 40, 50]
Using negative indexing: [30, 40, 50]
```

Thus, **negative indexing and slicing help access specific parts of a list easily**.

---

6 (b) Python Program to Convert Various Types of Data into List

Python provides the **list() function** to convert different data types such as **string, tuple, and set into a list**.

Program

```
# Converting string to list
s = "Python"
list1 = list(s)
print("String to List:", list1)

# Converting tuple to list
t = (1, 2, 3, 4)
list2 = list(t)
print("Tuple to List:", list2)

# Converting set to list
st = {10, 20, 30}
list3 = list(st)
print("Set to List:", list3)
```

Output

String to List: ['P', 'y', 't', 'h', 'o', 'n']
Tuple to List: [1, 2, 3, 4]
Set to List: [10, 20, 30]

Explanation

- list() function converts different data types into a **list format**.
- It works with **strings, tuples, sets, and other iterable objects**.

7 (a) Demonstrate the following string functions in Python with examples

Strings in Python have many **built-in functions and methods** used to manipulate text.

i) len()

The len() function returns the **length of a string**.

Example:

```
s = "Python"
print(len(s))
```

Output

6

---

ii) strip()

The strip() method **removes spaces from the beginning and end of a string**.

Example:

```
s = "  Hello  "
print(s.strip())
```

Output

Hello

---

iii) find()

The find() method **returns the index position of the first occurrence of a substring**.

Example:

```
s = "Python Programming"
print(s.find("Pro"))
```

Output

7

---

iv) split()

The split() method **splits a string into a list of words**.

Example:

```
s = "Python is easy"
print(s.split())
```

Output

['Python', 'is', 'easy']

---

v) index()

The index() method **returns the position of a substring in the string**.

Example:

```
s = "Hello Python"
print(s.index("Python"))
```

Output

6

---

7 (b) Python Program to Count Vowels, Consonants and Digits in a String

Program

```
s = input("Enter a string: ")

vowels = 0
consonants = 0
digits = 0

for ch in s:
  if ch.lower() in "aeiou":
    vowels += 1
  elif ch.isalpha():
    consonants += 1
  elif ch.isdigit():
    digits += 1

print("Vowels:", vowels)
print("Consonants:", consonants)
print("Digits:", digits)
```

Example Output

```
Enter a string: Python123
Vowels: 1
Consonants: 5
Digits: 3
```

Explanation

- The program checks each character in the string.
- If it is **a, e, i, o, u**, it is counted as a vowel.
- If it is an alphabet but not a vowel, it is a consonant.
- If it is a number, it is counted as a digit.

---

8 (a) Define Comprehension. Classify Comprehensions with Examples

Comprehension

Comprehension in Python is a **short and simple way to create collections (lists, sets, or dictionaries)** using a single line of code.

It makes programs **more readable and concise**.

Syntax

[expression for item in iterable if condition]

Types of Comprehensions

1. List Comprehension

Used to create a **list** in a compact way.

Example:

```
numbers = [x for x in range(5)]
print(numbers)
```

Output:

[0, 1, 2, 3, 4]

---

2. Set Comprehension

Used to create a **set**.

Example:

```
s = {x*x for x in range(5)}
print(s)
```

Output:

{0, 1, 4, 9, 16}

---

3. Dictionary Comprehension

Used to create a **dictionary**.

Example:

```
d = {x: x*x for x in range(5)}
print(d)
```

Output:

{0:0, 1:1, 2:4, 3:9, 4:16}

---

8 (b) Python Program to Generate Multiples of n Using List Comprehension

Program

```
n = int(input("Enter a number: "))

multiples = [n*i for i in range(1, 11)]

print("Multiples of", n, "are:")
print(multiples)
```

Example Output

```
Enter a number: 5
Multiples of 5 are:
[5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
```

Explanation

- The program uses **list comprehension**.
- range(1,11) generates numbers from 1 to 10.
- Each number is multiplied by **n** to generate multiples.
- The results are stored in a **list**.